

¡Hola!

Mi nombre es Tonatiuh Cruz. Me complace revisar tu proyecto hoy.

Al identificar cualquier error inicialmente, simplemente los destacaré. Te animo a localizar y abordar los problemas de forma independiente como parte de tu preparación para un rol como data-scientist. En un entorno profesional, tu líder de equipo seguiría un enfoque similar. Si encuentras la tarea desafiante, proporcionaré una pista más específica en la próxima iteración.

Encontrarás mis comentarios a continuación - **por favor no los muevas, modifiques o elimines**.

Puedes encontrar mis comentarios en cajas verdes, amarillas o rojas como esta:

Comentario del revisor

Éxito. Todo está hecho correctamente.

Comentario del revisor

Observaciones. Algunas recomendaciones.

Comentario del revisor

Necesita corrección. El bloque requiere algunas correcciones. El trabajo no puede ser aceptado con comentarios en rojo.

Puedes responderme utilizando esto:

Respuesta del estudiante. Hola Tonatiuh, nuevamente gracias por tus observaciones, creo que esta vez ya quedó.. y si tenias razon era mas sencillo de lo que pensaba.. Voy muy atrasada con el curso y de repente es frustrante no llevar el paso de la mayoría jeje pero agradezco la paciencia y ayuda, y aunque se me está complicando mas de lo que pensé, me gusta y lo disfruto. Quedo pendiente de cualquier obsrvación.

✓ ¡Llena ese carrito!

Review General. (Iteración 2)

Hola Carolina nuevamente! Felicitaciones porque has corregido los detalles marcados en nuestra iteración anterior. Ahora si este proyecto está en total condiciones de ser aprobado, bien hecho!

Éxitos en tu camino dentro del mundo de los datos y saludos!

Review General.

Caro, quería tomarme este tiempo al inicio de tu proyecto para comentarte mis apreciaciones generales de esta iteración de tu entrega.

Siempre me gusta comenzar dando la bienvenida al mundo de los datos a los estudiantes, te deseo lo mejor y espero que consigas lograr tus objetivos. Personalmente me gusta brindar el

siguiente consejo, "Está bien equivocarse, es normal y es lo mejor que te puede pasar. Aprendemos de los errores y eso te hará mejor programador ya que podrás descubrir cosas a medida que avances y son estas cosas las que te darán esa experiencia para ser un gran analista de datos."

Ahora si yendo a esta notebook. Caro quiero felicitarte porque has tenido un gran desempeño a lo largo del trabajo, has resuelto con creces todos los problemas planteados y te has destecado por tu manejo sobre las herramientas, muy bien hecho! Hemos tenido 2 detalles que deberíamos tratar de resolver y que estoy seguro que te tomará unos minutos, te he dejado los comentarios rojos con el contexto correspondiente para que puedas guiarte. En uno de los comentarios profundizamos en el penultimo ejercicio que mencionas :)

En resumen un gran proyecto pero que debemos corregir unos detalles!

Saludos y espero con ansias a nuestra próxima iteración Caro!

✓ Introducción

Instacart es una plataforma de entregas de comestibles donde la clientela puede registrar un pedido y hacer que se lo entreguen, similar a Uber Eats y Door Dash. El conjunto de datos que te hemos proporcionado tiene modificaciones del original. Redujimos el tamaño del conjunto para que tus cálculos se hicieran más rápido e introdujimos valores ausentes y duplicados. Tuvimos cuidado de conservar las distribuciones de los datos originales cuando hicimos los cambios.

Debes completar tres pasos. Para cada uno de ellos, escribe una breve introducción que refleje con claridad cómo pretendes resolver cada paso, y escribe párrafos explicatorios que justifiquen tus decisiones al tiempo que avanzas en tu solución. También escribe una conclusión que resuma tus hallazgos y elecciones.

✓ Diccionario de datos

Hay cinco tablas en el conjunto de datos, y tendrás que usarlas todas para hacer el preprocesamiento de datos y el análisis exploratorio de datos. A continuación se muestra un diccionario de datos que enumera las columnas de cada tabla y describe los datos que contienen.

- `instacart_orders.csv` : cada fila corresponde a un pedido en la aplicación Instacart.
 - `'order_id'` : número de ID que identifica de manera única cada pedido.
 - `'user_id'` : número de ID que identifica de manera única la cuenta de cada cliente.
 - `'order_number'` : el número de veces que este cliente ha hecho un pedido.
 - `'order_dow'` : día de la semana en que se hizo el pedido (0 si es domingo).
 - `'order_hour_of_day'` : hora del día en que se hizo el pedido.

- 'days_since_prior_order' : número de días transcurridos desde que este cliente hizo su pedido anterior.
- products.csv : cada fila corresponde a un producto único que pueden comprar los clientes.
 - 'product_id' : número ID que identifica de manera única cada producto.
 - 'product_name' : nombre del producto.
 - 'aisle_id' : número ID que identifica de manera única cada categoría de pasillo de víveres.
 - 'department_id' : número ID que identifica de manera única cada departamento de víveres.
- order_products.csv : cada fila corresponde a un artículo pedido en un pedido.
 - 'order_id' : número de ID que identifica de manera única cada pedido.
 - 'product_id' : número ID que identifica de manera única cada producto.
 - 'add_to_cart_order' : el orden secuencial en el que se añadió cada artículo en el carrito.
 - 'reordered' : 0 si el cliente nunca ha pedido este producto antes, 1 si lo ha pedido.
- aisles.csv
 - 'aisle_id' : número ID que identifica de manera única cada categoría de pasillo de víveres.
 - 'aisle' : nombre del pasillo.
- departments.csv
 - 'department_id' : número ID que identifica de manera única cada departamento de víveres.
 - 'department' : nombre del departamento.

Comentario del revisor

¡Hola! Excelente trabajo desarrollando la introducción y el diccionario de datos. Esto es crucial para cualquier proyecto, ya que establece una guía clara sobre los pasos a seguir. Tener estos elementos bien definidos desde el principio nos permite trabajar de manera más organizada y eficiente. En un futuro lo podrías complementar con una tabla de contenido.

✓ Paso 1. Descripción de los datos

Lee los archivos de datos ([/datasets/instacart_orders.csv](#) , [/datasets/products.csv](#) , [/datasets/aisles.csv](#) , [/datasets/departments.csv](#) y [/datasets/order_products.csv](#))

con `pd.read_csv()` usando los parámetros adecuados para leer los datos correctamente. Verifica la información para cada DataFrame creado.

✓ Plan de solución

Escribe aquí tu plan de solución para el Paso 1. Descripción de los datos.

1. Se llama a la librería de pandas
2. Guardar los data frames en variables
3. Mostrar información de los datos (`head()`), para saber con que vamos a trabajar y como manipularlos
4. Generar el formato correcto para ordenar los datos
5. Mostrar datos con `info()`

```
1 import pandas as pd
2 import numpy as np
3 from matplotlib import pyplot as plt
4 # importar librerías

1 df_orders = pd.read_csv('/datasets/instacart_orders.csv', sep= ";")
2 df_products = pd.read_csv('/datasets/products.csv', sep= ";")
3 df_aisles = pd.read_csv('/datasets/aisles.csv', sep= ";")
4 df_departments = pd.read_csv('/datasets/departments.csv', sep= ";")
5 df_order_products = pd.read_csv('/datasets/order_products.csv', sep= ";")# leer cc
```

Comentario del revisor:

Has realizado un excelente trabajo al importar los datos y las bibliotecas necesarias.

```
1 df_orders.head()
2 df_orders.info()# mostrar información del DataFrame
```

```
➡ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 478967 entries, 0 to 478966
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   order_id                             478967 non-null  int64
1   user_id                              478967 non-null  int64
2   order_number                         478967 non-null  int64
3   order_dow                            478967 non-null  int64
4   order_hour_of_day                    478967 non-null  int64
5   days_since_prior_order               450148 non-null  float64
dtypes: float64(1), int64(5)
memory usage: 21.9 MB
```

```
1 print(df_products.head())
2 print(df_products.info())# mostrar información del DataFrame
```

```

⇒ product_id      product_name  aisle_id \
0          1      Chocolate Sandwich Cookies      61
1          2      All-Seasons Salt      104
2          3      Robust Golden Unsweetened Oolong Tea      94
3          4      Smart Ones Classic Favorites Mini Rigatoni Wit...      38
4          5      Green Chile Anytime Sauce      5

      department_id
0          19
1          13
2           7
3           1
4          13
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49694 entries, 0 to 49693
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   product_id      49694 non-null   int64
1   product_name    48436 non-null   object
2   aisle_id        49694 non-null   int64
3   department_id   49694 non-null   int64
dtypes: int64(3), object(1)
memory usage: 1.5+ MB
None
```

```
1 print(df_aisles.head())
2 print(df_aisles.info())# mostrar información del DataFrame
```

```

⇒ aisle_id      aisle
0          1      prepared soups salads
1          2      specialty cheeses
2          3      energy granola bars
3          4      instant foods
4          5      marinades meat preparation
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 134 entries, 0 to 133
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   aisle_id      134 non-null   int64
1   aisle         134 non-null   object
dtypes: int64(1), object(1)
memory usage: 2.2+ KB
None
```

```
1 print(df_departments.head())
2 print(df_departments.info())# mostrar información del DataFrame
```

```

➡ department_id department
0          1      frozen
1          2       other
2          3       bakery
3          4       produce
4          5       alcohol
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   department_id    21 non-null    int64
1   department        21 non-null    object
dtypes: int64(1), object(1)
memory usage: 464.0+ bytes
None

```

```

1 print(df_order_products.head())
2 print(df_order_products.info())# mostrar información del DataFrame

```

```

➡   order_id  product_id  add_to_cart_order  reordered
0    2141543      11440             17.0          0
1     567889      1560              1.0          1
2    2261212     26683              1.0          1
3     491251      8670             35.0          1
4    2571142      1940              5.0          1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4545007 entries, 0 to 4545006
Data columns (total 4 columns):
#   Column              Dtype
---  -
0   order_id            int64
1   product_id          int64
2   add_to_cart_order   float64
3   reordered           int64
dtypes: float64(1), int64(3)
memory usage: 138.7 MB
None

```

Comentario del revisor:

Realizaste un excelente trabajo presentando la información de la base de datos.

Conclusiones

Fue necesario utilizar el argumento, sep= con el separador correcto, para ordenar los datos y poder comenzar a manipularlos. Se identifican algunos valores ausentes en 3 de las tablas.

✓ Paso 2. Preprocesamiento de los datos

Preprocesa los datos de la siguiente manera:

- Verifica y corrige los tipos de datos (por ejemplo, asegúrate de que las columnas de ID sean números enteros).
- Identifica y completa los valores ausentes.
- Identifica y elimina los valores duplicados.

Asegúrate de explicar qué tipos de valores ausentes y duplicados encontraste, cómo los completaste o eliminaste y por qué usaste esos métodos. ¿Por qué crees que estos valores ausentes y duplicados pueden haber estado presentes en el conjunto de datos?

Plan de solución

Escribe aquí tu plan para el Paso 2. Preprocesamiento de los datos.

1. Después de revisar tipo de datos valorar si es necesario convertir las columnas en enteros
2. identificar si hay valores duplicado y eliminarlos
3. revisar si hay valores ausentes utilizando `isnull().sum()` y valorar si se pueden eliminar o sustituir valores.

✓ Encuentra y elimina los valores duplicados (y describe cómo tomaste tus decisiones).

✓ orders data frame

```
1 df_orders['order_id'].duplicated().sum() # Revisa si hay pedidos duplicados
2
```

↔ 15

```
1 df_orders[df_orders['order_id'].duplicated()]
```



	order_id	user_id	order_number	order_dow	order_hour_of_day	days_since_prior_
145574	794638	50898	24	3	2	
223105	2160484	107525	16	3	2	
230807	1918001	188546	14	3	2	
266232	1782114	106752	1	3	2	
273805	1112182	202304	84	3	2	
284038	2845099	31189	11	3	2	
311713	1021560	53767	3	3	2	
321100	408114	68324	4	3	2	
323900	1919531	191501	32	3	2	
345917	2232988	82565	1	3	2	
371905	391768	57671	19	3	2	
394347	467134	63189	21	3	2	
411408	1286742	183220	48	3	2	
415163	2282673	86751	49	3	2	
441599	2125197	14050	48	3	2	

¿Tienes líneas duplicadas? Si sí, ¿qué tienen en común?

```
1 # Basándote en tus hallazgos,
2 df_orders_wednesday= df_orders[(df_orders['order_dow'] == 3) & (df_orders['order_id']
3
```

```
1 df_orders_wednesday
```



	order_id	user_id	order_number	order_dow	order_hour_of_day	days_since_prior_
4838	2766110	162084	41	3		2
5156	2190225	138285	18	3		2
15506	553049	58599	13	3		2
18420	382357	120200	19	3		2
24691	690242	77357	2	3		2
...	...	...	...	...		...
457013	3384021	14881	6	3		2
458816	910166	164782	18	3		2
459635	1680532	106435	6	3		2
468324	222962	54979	59	3		2
477526	2592344	46860	38	3		2

121 rows × 6 columns



1 Comienza a programar o [generar](#) con IA.

¿Qué sugiere este resultado? Que quizá hubo algún error en el sistema el día miércoles a las 2 am

```
1 df_orders.drop_duplicates(inplace= True)
2 df_orders.reset_index(inplace=True,drop=True)# Elimina los pedidos duplicados
3
```

```
1 df_orders.duplicated().sum()# Vuelve a verificar si hay filas duplicadas
2
```

0

```
1 df_orders['order_id'].duplicated().sum()# Vuelve a verificar únicamente si hay IDs
2
```

0

Describe brevemente tus hallazgos y lo que hiciste con ellos.

En el dataframe de orders se encontraron valores duplicados en los pedido que coincidían en una determinada hora y día. Se eliminaron los valores duplicados y se verificó que ya no hubiera.

Comentario del revisor

Muy buen trabajo!! Desarrollaste de manera excelente el análisis de duplicados y eliminaste esos casos.

▼ products data frame

```
1 df_products.duplicated().sum()# Verifica si hay filas totalmente duplicadas
2
```

⇒ 0

```
1 df_products['department_id'].drop_duplicates()# )# Revisa únicamente si hay ID de
2
```

⇒

0	19
1	13
2	7
3	1
5	11
8	16
13	17
14	18
16	12
18	9
20	8
27	14
28	15
30	4
37	21
46	6
48	20
51	5
57	3
85	2
502	10

Name: department_id, dtype: int64

```
1 df_products_uppered= df_products['product_name'].str.upper()# Revisa únicamente si
2 products_uppered.duplicated().sum()
```

⇒ 0

```
1 df_products_uppered.drop_duplicates(inplace=True)
```

```
1 df_products_uppered.duplicated().sum()
2
```

 0

```
1 df_products['product_uppear']= df_products_uppered
2 df_products
```



	product_id	product_name	aisle_id	department_id	product_uppear
0	1	Chocolate Sandwich Cookies	61	19	CHOCOLATE SANDWICH COOKIES
1	2	All-Seasons Salt	104	13	ALL-SEASONS SALT
2	3	Robust Golden Unsweetened Oolong Tea	94	7	ROBUST GOLDEN UNSWEETENED OOLONG TEA
3	4	Smart Ones Classic Favorites Mini Rigatoni Wit...	38	1	SMART ONES CLASSIC FAVORITES MINI RIGATONI WIT...
4	5	Green Chile Anytime Sauce	5	13	GREEN CHILE ANYTIME SAUCE
...	...	...	...	...	...
49689	49690	HIGH PERFORMANCE ENERGY DRINK	64	7	NaN
49690	49691	ORIGINAL PANCAKE & WAFFLE MIX	130	14	NaN

```
1 duplicate_product_names= df_products[(~df_products['product_name'].isna()) & df_pr
```

```
1 duplicate_product_names # ESTE RESULTADO DEBERÍAMOS VER
```



	product_id	product_name	aisle_id	department_id	product_upper
2058	2059	Biotin 1000 Mcg	47	11	NaN
5455	5456	Green Tea With Ginseng and Honey	96	20	NaN
5558	5559	Cream Of Mushroom Soup	69	15	NaN
7558	7559	Cinnamon Rolls with Icing	105	13	NaN
9037	9038	American Cheese slices	21	16	NaN
...	...	...	...	...	...
49689	49690	HIGH PERFORMANCE ENERGY DRINK	64	7	NaN
49690	49691	ORIGINAL PANCAKE & WAFFLE MIX	130	14	NaN
49691	49692	ORGANIC INSTANT OATMEAL LIGHT MAPLE BROWN SUGAR	130	14	NaN
49692	49693	SPRING WATER BODY WASH	127	11	NaN

Comentario del revisor

Excelente trabajo! Solamente la verificación de duplicados te recomiendo hacerla sobre la variable de nombre del producto. Para esto te puedes guiar del siguiente ejemplo:

```
products[(~products['product_name'].isna()) & products['product_name'].duplicated()]
```

Comentario del revisor

Ya casi Caro! Tenemos todo lo necesario para aplicar esta última corrección, nosotros calculamos el nombre de producto en mayúscula y el objetivo es corroborar en base a este valor los duplicados ya que al estar todos en mayúsculas podemos corroborarlo de forma uniforme, pero a este valor lo guardamos en un nuevo dataframe, te recomiendo que a la columna transformada en mayúscula la agregues a df_products y luego en el cálculo de duplicados verificar justamente sobre product_upper a los duplicados

```
..... & df_products['product_upper'].duplicated()]
```

deberíamos ver 104 casos

Ahora si felicitaciones Carolina!

Describe brevemente tus hallazgos y lo que hiciste con ellos.

Se examinó el dataframe de "products" para identificar posibles elementos duplicados. Para ello, se estandarizó el formato de la columna de product_name, convirtiendo todo a mayúsculas para facilitar la comparación. Se detectaron duplicados, los cuales fueron eliminados para mejorar el análisis

✓ departments data frame

```
1 df_departments.duplicated().sum()# Revisa si hay filas totalmente duplicadas
```

➡ 0

```
1 df_departments['department_id'].duplicated().sum()# Revisa únicamente si hay IDs c
```

➡ 0

Describe brevemente tus hallazgos y lo que hiciste con ellos.

Se utilizó el metodo duplicated() para buscar filas duplicadas y para buscar duplicados en la columna 'department_id', y no se encontraron valores duplicados

✓ aisles data frame

```
1 df_aisles.duplicated().sum()# Revisa si hay filas totalmente duplicadas
```

➡ 0

```
1 df_aisles['aisle_id'].duplicated().sum()# Revisa únicamente si hay IDs duplicadas
```

➡ 0

Describe brevemente tus hallazgos y lo que hiciste con ellos En el dataframe de aisles, no se encontraron duplicados. Se utilizó el metrodo duplicated().sum() al dataframe y a una de sus columnas

✓ order_products data frame

```
1 df_order_products.duplicated().sum()# Revisa si hay filas totalmente duplicadas
2
```

0

```
1 df_order_products# Vuelve a verificar si hay cualquier otro duplicado engañoso
2
```

	order_id	product_id	add_to_cart_order	reordered
0	2141543	11440	17.0	0
1	567889	1560	1.0	1
2	2261212	26683	1.0	1
3	491251	8670	35.0	1
4	2571142	1940	5.0	1
...	...	...	...	...
4545002	577211	15290	12.0	1
4545003	1219554	21914	9.0	0
4545004	692640	47766	4.0	1
4545005	319435	691	8.0	1
4545006	1398151	28733	9.0	0

4545007 rows × 4 columns

Describe brevemente tus hallazgos y lo que hiciste con ellos.

✓ Encuentra y elimina los valores ausentes

Al trabajar con valores duplicados, pudimos observar que también nos falta investigar valores ausentes:

- La columna 'product_name' de la tabla products.
- La columna 'days_since_prior_order' de la tabla orders.
- La columna 'add_to_cart_order' de la tabla order_productos.

✓ products data frame

```
1 missing_df_products= df_products[df_products['product_name'].isnull()]# Encuentra
```

```
1 missing_df_products
```

```
2
```



	product_id	product_name	aisle_id	department_id
37	38	NaN	100	21
71	72	NaN	100	21
109	110	NaN	100	21
296	297	NaN	100	21
416	417	NaN	100	21
...	...	...	...	...
49552	49553	NaN	100	21
49574	49575	NaN	100	21
49640	49641	NaN	100	21
49663	49664	NaN	100	21
49668	49669	NaN	100	21

1258 rows × 4 columns

Describe brevemente cuáles son tus hallazgos. Al utilizar este código se filtran las filas de la columna que contienen valores ausentes, y después se imprimen... Se encontraron 1258 filas con valores ausentes(NaN)

```
1 missing_products= df_products[df_products['product_name'].isnull()]['aisle_id'] #
```

```
1 missing_id_100 = missing_products.unique() == 100
```

```
1 missing_id_100
```



```
array([ True])
```

Describe brevemente cuáles son tus hallazgos. Todos los valores ausentes están relacionados con el pasillo ID 100.

```
1 missing_by_department = df_products[df_products['product_name'].isnull()]['departm
```

```
2
```

```
1 id_department = missing_by_department.unique() == 21
```

```
1 if id_department:
```

```
2     print("Todos los nombres de productos ausentes están relacionados con el depar
```

```
3 else:
```

```
4     print("No todos los nombres de productos ausentes están relacionados con el de
```

➡ Todos los nombres de productos ausentes están relacionados con el departamento con ID 21



Describe brevemente cuáles son tus hallazgos. Los nombresde productos ausentes estan ralacionados con el pasillo 21 y 100

```
1 aisle_100_data = df_aisles[df_aisles['aisle_id'] == 100]
```

```
2
```

```
3 department_21_data = df_products[df_products['department_id']== 21]
```

```
4                                     # Usa las tablas department y aisle para revisar
```

```
1 aisle_100_data
```

```
2
```

```
3
```



	aisle_id	aisle
99	100	missing

```
1 department_21_data
```



	product_id	product_name	aisle_id	department_id
37	38	NaN	100	21
71	72	NaN	100	21
109	110	NaN	100	21
296	297	NaN	100	21
416	417	NaN	100	21
...	...	...	...	...
49552	49553	NaN	100	21
49574	49575	NaN	100	21
49640	49641	NaN	100	21
49663	49664	NaN	100	21
49668	49669	NaN	100	21

1258 rows × 4 columns

Describe brevemente cuáles son tus hallazgos. Los nombres de productos del pasillo 100 y departamento producto esatan ausentes

```
1 df_products['product_name'].fillna('Unknown', inplace=True)# Completa los nombres
2 df_products['product_name'].head(50)
3
```



```
0 Chocolate Sandwich Cookies
1 All-Seasons Salt
2 Robust Golden Unsweetened Oolong Tea
3 Smart Ones Classic Favorites Mini Rigatoni Wit...
4 Green Chile Anytime Sauce
5 Dry Nose Oil
6 Pure Coconut Water With Orange
7 Cut Russet Potatoes Steam N' Mash
8 Light Strawberry Blueberry Yogurt
9 Sparkling Orange Juice & Prickly Pear Beverage
10 Peach Mango Juice
11 Chocolate Fudge Layer Cake
12 Saline Nasal Mist
13 Fresh Scent Dishwasher Cleaner
14 Overnight Diapers Size 6
15 Mint Chocolate Flavored Syrup
16 Rendered Duck Fat
17 Pizza for One Suprema Frozen Pizza
18 Gluten Free Quinoa Three Cheese & Mushroom Blend
19 Pomegranate Cranberry & Aloe Vera Enrich Drink
20 Small & Medium Dental Dog Treats
```

```

21             Fresh Breath Oral Rinse Mild Mint
22             Organic Turkey Burgers
23     Tri-Vi-Sol® Vitamins A-C-and D Supplement Drop...
24             Salted Caramel Lean Protein & Fiber Bar
25             Fancy Feast Trout Feast Flaked Wet Cat Food
26     Complete Spring Water Foaming Antibacterial Ha...
27             Wheat Chex Cereal
28     Fresh Cut Golden Sweet No Salt Added Whole Ker...
29             Three Cheese Ziti- Marinara with Meatballs
30             White Pearl Onions
31             Nacho Cheese White Bean Chips
32             Organic Spaghetti Style Pasta
33             Peanut Butter Cereal
34             Italian Herb Porcini Mushrooms Chicken Sausage
35     Traditional Lasagna with Meat Sauce Savory Ita...
36             Noodle Soup Mix With Chicken Broth
37             Unknown
38             Daily Tangerine Citrus Flavored Beverage
39     Beef Hot Links Beef Smoked Sausage With Chile ...
40             Organic Sourdough Einkorn Crackers Rosemary
41             Biotin 1000 mcg
42             Organic Clementines
43             Sparkling Raspberry Seltzer
44             European Cucumber
45             Raisin Cinnamon Bagels 5 count
46             Onion Flavor Organic Roasted Seaweed Snack
47             School Glue- Washable- No Run
48             Vegetarian Grain Meat Sausages Italian - 4 CT
49             Pumpkin Muffin Mix
Name: product_name, dtype: object

```

Describe brevemente tus hallazgos y lo que hiciste con ellos. Se utilizó el método fillna() para remplazar los valores ausentes NaN en la columna product_name, con la cadena Unknown

Comentario del revisor

Muy buen trabajo!! Desarrollaste de manera excelente el análisis de valores faltantes y los llenaste con "unknown".

✓ orders data frame

```
1 df_orders.isna().sum()# Encuentra los valores ausentes
```

```

➡ order_id          0
   user_id          0
   order_number     0
   order_dow        0
   order_hour_of_day 0
   days_since_prior_order 28817
   dtype: int64

```

```

1 missing_not_first_order = df_orders[(df_orders.isnull().any(axis=1)) & (df_orders[

1 if missing_not_first_order.empty:
2     print("No hay valores ausentes que no sean el primer pedido del cliente.")
3 else:
4     print("Hay valores ausentes que no son el primer pedido del cliente:")
5     print(missing_not_first_order)

```

➡ No hay valores ausentes que no sean el primer pedido del cliente.

Describe brevemente tus hallazgos y lo que hiciste con ellos. Se utilizó el método `isna()` para verificar si había valores ausentes en el dataframe `orders`, y se encontraron valores en la columna `days_since_prior_order`, ya que no hay valores ausentes que no sean el primer pedido

```

1 df_order_products ## `order_products` data frame

```



	order_id	product_id	add_to_cart_order	reordered
0	2141543	11440	17.0	0
1	567889	1560	1.0	1
2	2261212	26683	1.0	1
3	491251	8670	35.0	1
4	2571142	1940	5.0	1
...	...	...	...	...
4545002	577211	15290	12.0	1
4545003	1219554	21914	9.0	0
4545004	692640	47766	4.0	1
4545005	319435	691	8.0	1
4545006	1398151	28733	9.0	0

4545007 rows × 4 columns

```

1 df_order_products.isna().sum() # Encuentra los valores ausentes

```



```

order_id      0
product_id    0
add_to_cart_order  836
reordered      0
dtype: int64

```

```
1 df_order_products['add_to_cart_order'].min() # ¿Cuáles son los valores mínimos y m
2
```

```
➞ 1.0
```

```
1 df_order_products['add_to_cart_order'].max()
```

```
➞ 64.0
```

Describe brevemente cuáles son tus hallazgos. se encontraron 836 valores ausentes en el dataframe en la columna "add_to_cart_order". Se obtuvieron valores minimos y maximos de esa columna lo que nos da una idea de la distribucion de los datos, en este caso el rango de productos que se agregan al carrito

```
1 missing_orders= df_order_products[df_order_products['add_to_cart_order'].isnull()]
2 missing_order_ids = missing_orders.tolist()
3 missing_order_ids
```

```
➞ [2449164,
1968313,
2926893,
1717990,
1959075,
844733,
61355,
936852,
264710,
1717990,
1564093,
129627,
264710,
293169,
2849370,
1386261,
3308010,
903110,
2136777,
3347453,
1888628,
165801,
2449164,
2094761,
3347453,
1038146,
2136777,
2997021,
844733,
844733,
2136777,
813364,
165801,
```

2256933,
 171934,
 264710,
 2449164,
 2409109,
 3347453,
 813364,
 2926893,
 936852,
 1730767,
 1169835,
 733526,
 844733,
 293169,
 61355,
 903110,
 1968313,
 733526,
 2136777,
 2926893,
 404157,
 1386261,
 171934,
 3125735,
 747668,

```
1 missing_orders_id = df_order_products[(df_order_products['add_to_cart_order']).isna()
2 orders_above_64 = (missing_orders_id > 64).all()# ¿Todos los pedidos con valores a
3 grouped_orders = missing_orders_id.groupby('order_id')['product_id'].count()# Agru
4 min_products = grouped_orders.min() # Cuenta el número de 'product_id' en cada pec
5 print(orders_above_64)
6 print(grouped_orders)
7 print(min_products)
8
```

```
⇒ order_id      True
   product_id   False
   add_to_cart_order False
   reordered     False
   dtype: bool
   order_id
   9310      1
   61355     63
   102236    31
   129627     5
   165801     6
   ..
   2999801    6
   3125735    22
   3308010    51
   3347453     7
   3383594     5
   Name: product_id, Length: 70, dtype: int64
1
```

Describe brevemente cuáles son tus hallazgos. la columna insta_cart_order tiene valores ausentes, el orden en el que se ghan agreagdo los productos osocilan entre 1 y 63 productos

```
1 df_order_products['add_to_cart_order'] = df_order_products['add_to_cart_order'].fillna(0)
2 df_order_products['add_to_cart_order'] = df_order_products['add_to_cart_order'].astype(int)
3 df_order_products['add_to_cart_order']
4
```

```
0      17
1       1
2       1
3      35
4       5
..
4545002  12
4545003   9
4545004   4
4545005   8
4545006   9
Name: add_to_cart_order, Length: 4545007, dtype: int64
```

Describe brevemente tus hallazgos y lo que hiciste con ellos.

Conclusiones

Escribe aquí tus conclusiones intermedias sobre el Paso 2. Preprocesamiento de los datos

Se llevó a cabo un proceso exhaustivo de limpieza de datos con el objetivo de eliminar cualquier posible sesgo que pudiera influir en el análisis y comprometer la calidad de los resultados obtenidos. Durante este proceso, se identificaron algunos valores duplicados que, para asegurar la integridad del conjunto de datos, fueron eliminados de manera cuidadosa. Además, se encontraron datos ausentes; en ciertos casos, estos se reemplazaron por la cadena 'unknown', especialmente en lo que respecta a los nombres de productos.

La limpieza y el procesamiento de datos son pasos fundamentales para asegurar que la información sea precisa y confiable. Este esfuerzo no solo mejora la calidad de los datos, sino que también facilita un análisis más efectivo y significativo, permitiendo que se tomen decisiones más informadas basadas en resultados sólidos. Sin un adecuado tratamiento de los datos, el riesgo de obtener conclusiones erróneas se incrementa, lo que subraya la importancia de realizar este tipo de tareas de forma meticulosa.

Paso 3. Análisis de los datos

Una vez los datos estén procesados y listos, haz el siguiente análisis:

✓ [A] Fácil (deben completarse todos para aprobar)

1. Verifica que los valores en las columnas 'order_hour_of_day' y 'order_dow' en la tabla orders sean razonables (es decir, 'order_hour_of_day' oscile entre 0 y 23 y 'order_dow' oscile entre 0 y 6).
2. Crea un gráfico que muestre el número de personas que hacen pedidos dependiendo de la hora del día.
3. Crea un gráfico que muestre qué día de la semana la gente hace sus compras.
4. Crea un gráfico que muestre el tiempo que la gente espera hasta hacer su siguiente pedido, y comenta sobre los valores mínimos y máximos.

✓ [A1] Verifica que los valores sean sensibles

```
1 df_orders['order_hour_of_day'].min(), df_orders['order_hour_of_day'].max()
```

```
➡ (0, 23)
```

```
1 df_orders['order_dow'].min(), df_orders['order_dow'].max()
```

```
➡ (0, 6)
```

Escribe aquí tus conclusiones Los varoles de ambas columnas son razonables. las horas del dia oscilan entre 1 y 23 horas ya que hay 24 horas en el día y para el caso de order_dow, osila entre 0 y 6 ya que son 7 dias de de la semana, es necesario verificar para evitar posibles errores.

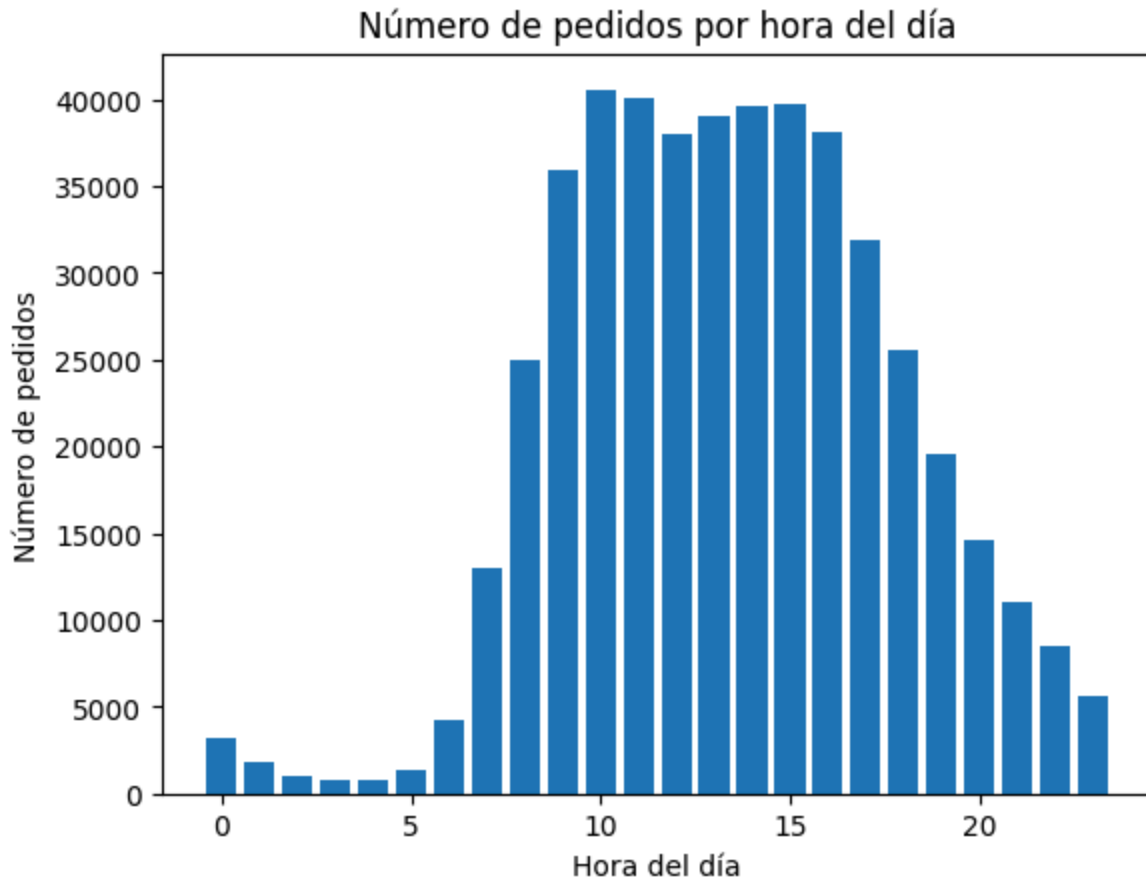
Comentario Revisor

Muy buena forma de revisar que los valores sean sensibles. Otra forma de hacerlo que puedes explorar más adelante es el uso de la función `.unique()` o `.value_counts()`.

✓ [A2] Para cada hora del día, ¿cuántas personas hacen órdenes?

```
1 orders_by_hour = df_orders.groupby('order_hour_of_day')['order_id'].count()
2 plt.bar(orders_by_hour.index, orders_by_hour.values)
3 plt.xlabel('Hora del día')
4 plt.ylabel('Número de pedidos')
```

```
5 plt.title('Número de pedidos por hora del día')  
6 plt.show()
```



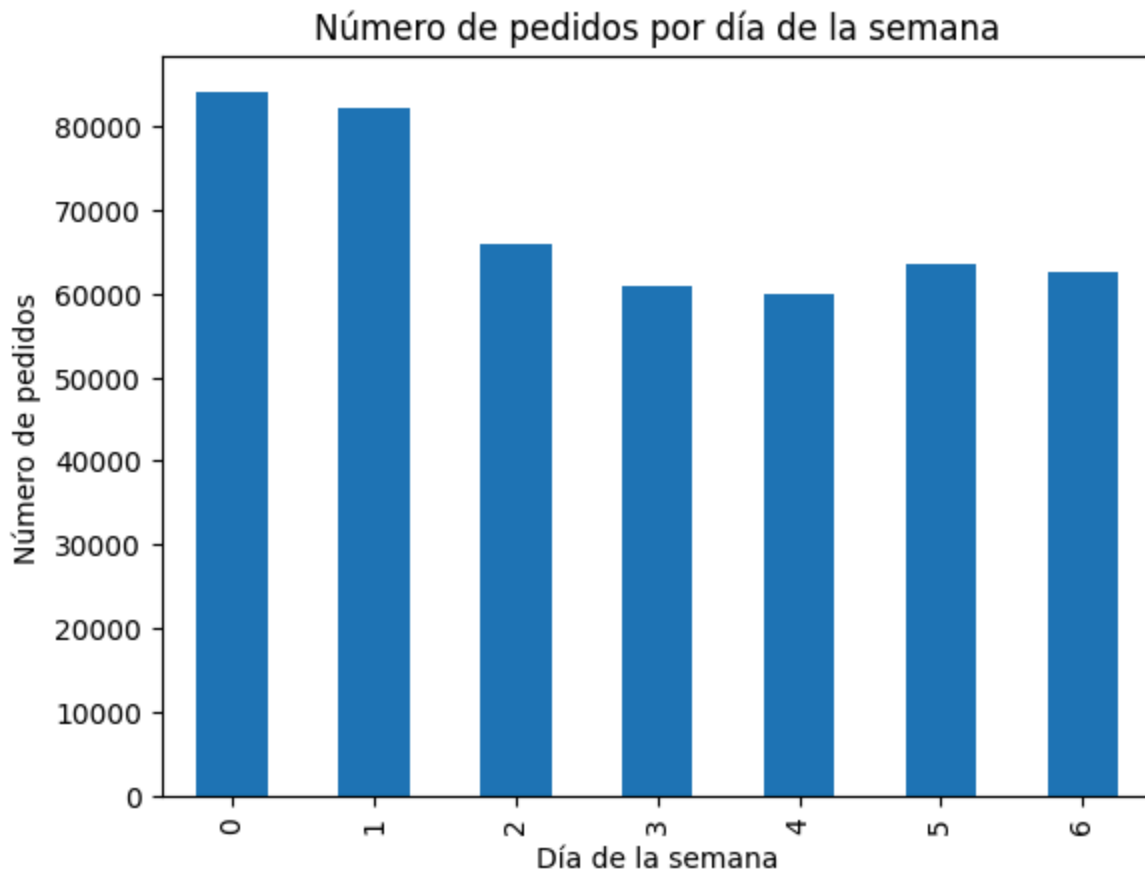
Escribe aquí tus conclusiones: La mayor cantidad de pedidos se realizan entre 10 de la mañana y 3 de la tarde con un maximo de 4000 personas por día.

Comentario revisor

Muy buen trabajo el desarrollar una gráfica para visualizar los resultados

✓ [A3] ¿Qué día de la semana compran víveres las personas?

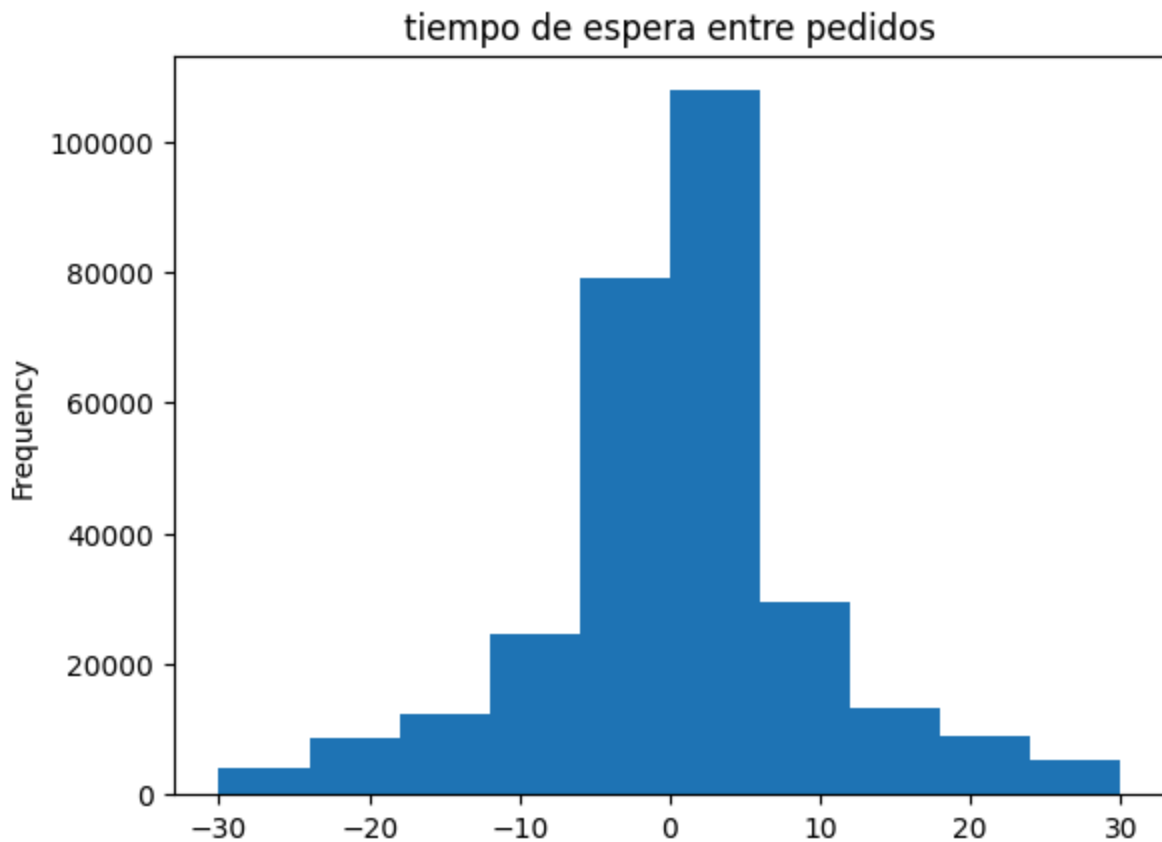
```
1 orders_by_dow = df_orders.groupby('order_dow')['order_id'].count()  
2 orders_by_dow.plot(kind= "bar",xlabel='Día de la semana',ylabel='Número de pedidos'  
3 plt.show()
```



Escribe aquí tus conclusiones: El día que mas se realizan pedidos en la semana es el día domingo con mas de 8000 pedidos seguido del día lunes.

✓ [A4] ¿Cuánto tiempo esperan las personas hasta hacer otro pedido?
Comenta sobre los valores mínimos y máximos.

```
1 orders = df_orders.sort_values(['user_id', 'order_dow'])
2 orders_day_since_prior_order= orders.groupby('user_id')['days_since_prior_order'].
3 orders_day_since_prior_order.plot(kind= 'hist', xlabel= 'dias desde el pedido ante
4 plt.show()
```



Escribe aquí tus conclusiones: la mayor cantidad de pedidos se realiza no rebaza los 5 días posteriores a su primer pedido

✓ [B] Intermedio (deben completarse todos para aprobar)

1. ¿Existe alguna diferencia entre las distribuciones 'order_hour_of_day' de los miércoles y los sábados? Traza gráficos de barra de 'order_hour_of_day' para ambos días en la misma figura y describe las diferencias que observes.
2. Grafica la distribución para el número de órdenes que hacen los clientes (es decir, cuántos clientes hicieron solo 1 pedido, cuántos hicieron 2, cuántos 3, y así sucesivamente...).
3. ¿Cuáles son los 20 principales productos que se piden con más frecuencia (muestra su identificación y nombre)?

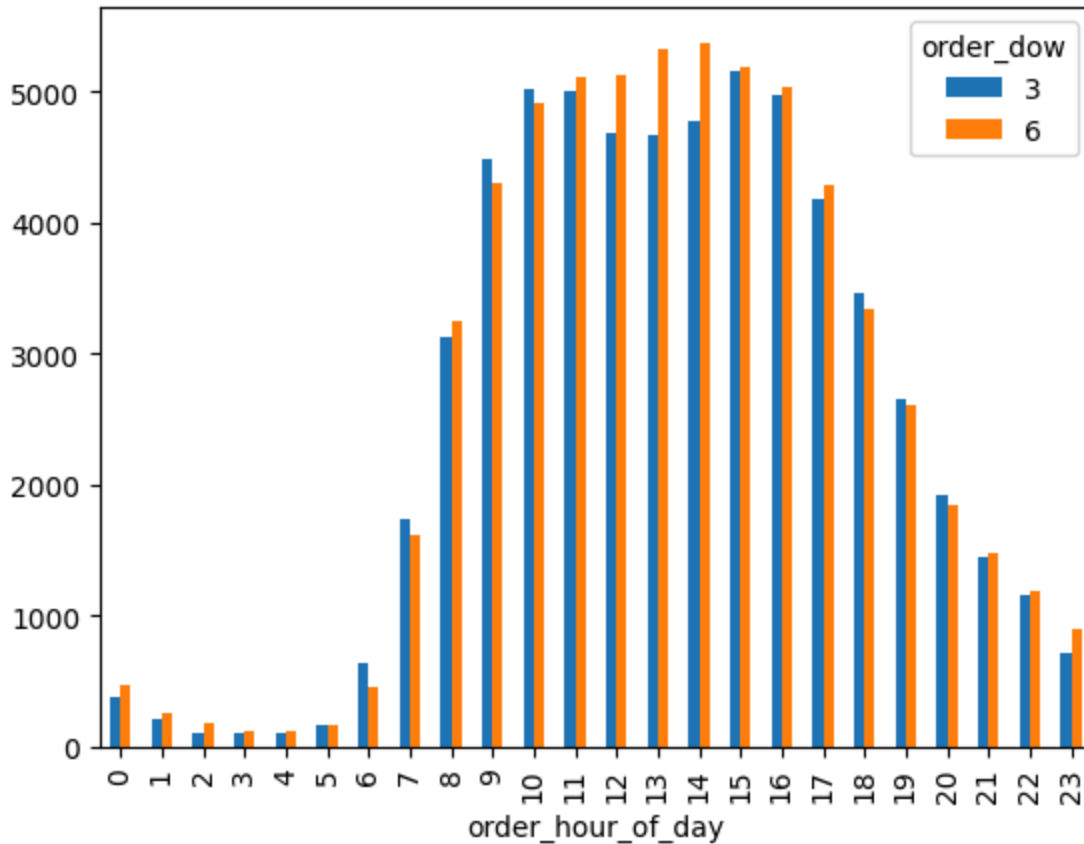
[B1] Diferencia entre miércoles y sábados para 'order_hour_of_day'.

- ✓ Traza gráficos de barra para los dos días y describe las diferencias que veas.

```
1 orders_hours_filter= df_orders[(df_orders['order_dow']== 3)| (df_orders['order_dow']
```

```
1 orders_hours_filter.pivot_table(index='order_hour_of_day',columns= 'order_dow',va]
```

```
>>> <AxesSubplot:xlabel='order_hour_of_day'>
```



1 Comienza a programar o generar con IA.

1 Comienza a programar o generar con IA.

Escribe aquí tus conclusiones: Las personas usualmente realizan mayor cantidad de pedidos los días sábados que los días miércoles, principalmente entre las 11 am y 5 pm.

los días miércoles entre 9 y 10 de la mañana tienen mayor cantidad de pedidos que los días sábados. los horarios en que se realizan menos pedidos son entre la 1 y 5 de la mañana para ambos días

✓ [B2] ¿Cuál es la distribución para el número de pedidos por cliente?

```
1 order_counts = df_orders['user_id'].value_counts()
```

```
1 plt.hist(order_counts, bins=range(1, order_counts.index.max() + 2))
2 plt.xlabel('Número de órdenes')
3 plt.ylabel('Número de clientes')
4 plt.title('Distribución del número de órdenes por cliente')
5 plt.show()
```

1 Comienza a programar o generar con IA.

Escribe aquí tus conclusiones

✓ [B3] ¿Cuáles son los 20 productos más populares (muestra su ID y nombre)?

```
1 products_merge= df_products.merge(df_order_products, on= 'product_id', how= 'outer')

1 products_filter= products_merge.groupby(['product_id', 'product_name'])['order_id'].count()

1 products_filter.head(20)
```

Escribe aquí tus conclusiones: El producto mas popular es es la banana, por lo que se puede intuir la predilección por este producto ya sea por sabor o costo. Los productos organicos tienen mucha demanda. las frutas y verduras son los alimentos mas buscados

Comentario del revisor

Muy buen desarrollo de la sección. Desarrollaste de buena manera todos los análisis y lo complementaste con una gráfica.

✓ [C] Difícil (deben completarse todos para aprobar)

1. ¿Cuántos artículos suelen comprar las personas en un pedido? ¿Cómo es la distribución?
2. ¿Cuáles son los 20 principales artículos que vuelven a pedirse con mayor frecuencia (muestra sus nombres e IDs de los productos)?
3. Para cada producto, ¿cuál es la tasa de repetición del pedido (número de repeticiones de pedido/total de pedidos)?
4. Para cada cliente, ¿qué proporción de los productos que pidió ya los había pedido? Calcula la tasa de repetición de pedido para cada usuario en lugar de para cada producto.

5. ¿Cuáles son los 20 principales artículos que la gente pone primero en sus carritos (muestra las IDs de los productos, sus nombres, y el número de veces en que fueron el primer artículo en añadirse al carrito)?

✓ [C1] ¿Cuántos artículos compran normalmente las personas en un pedido?
¿Cómo es la distribución?

```
1 items_per_order= df_order_products.groupby('order_id')['product_id'].agg(['count'])

1 items_per_order.describe()

1 plt.hist(items_per_order['count'], bins=range(1, items_per_order['count'].max() ))
2 plt.xlabel('Número de artículos por pedido')
3 plt.ylabel('Número de pedidos')
4 plt.title('Distribución del número de artículos por pedido')
5 plt.show()
```

Escribe aquí tus conclusiones: La mayoría de pedidos oscilan entre 1 y 10 articulo por pedido. La mayoría de personas tiene un promedio de 10 articulos por pedido. Menos del 20% de los perdidos ordena mas de 20 articulos

✓ [C2] ¿Cuáles son los 20 principales artículos que vuelven a pedirse con mayor frecuencia (muestra sus nombres e IDs de los productos)?

```
1 reorder_merge= products_merge[products_merge['reordered']== 1]

1 reorder_filter= reorder_merge.groupby(['product_id','product_name'])['order_id'].c

1 reorder_filter.head(20)

1 Comienza a programar o generar con IA.
```

Escribe aquí tus conclusiones: Los 20 articulos que vuelven a pedirse con mayor frecuencia son las frutas y verduras organicos, siendo la banana la primera opcion.

✓

[C3] Para cada producto, ¿cuál es la proporción de las veces que se pide y que se vuelve a pedir?

```
1 product_orders = df_order_products.groupby('product_id')['order_id'].agg(['count'])

1 product_reorders= product_orders[product_orders['count'] > 1]

1 reoder_rates= product_reorders['count'] / product_orders['count']
2 reoder_rates
```

Escribe aquí tus conclusiones

✓ [C4] Para cada cliente, ¿qué proporción de sus productos ya los había pedido?

```
1 df_merge= df_orders.merge(df_order_products)
2

1 df_merge.groupby('user_id')['reordered'].mean()
```

```
⇒ user_id
2      0.038462
4      0.000000
5      0.666667
6      0.000000
7      0.928571
...
206203  0.222222
206206  0.714286
206207  0.891304
206208  0.696000
206209  0.320000
Name: reordered, Length: 149626, dtype: float64
```

1 Comienza a programar o generar con IA.

Comentario del revisor

Muy buena utilización de la agrupación de los valores. Pero en este caso nos piden sacar el dato para cada cliente, en este sentido dentro del groupby te sugiero utilizar la variable 'user_id' y en este caso el ejercicio hace referencia a cuál es la proporción de veces que se vuelve a pedir cada producto. Es por ello que la variable que nos interesa hacer el mean es ['reordered']

```
1 repeat_orders_by_user
```

Comentario del revisor

Un buen acercamiento pero estamos cometiendo un leve error, a este punto lo podríamos resolver de forma más sencilla, antes que nada si queremos ver las ordenes y las repeticiones de pedidos deberíamos unificar los dataframe que contienen la información en uno mismo, es decir

```
df_orders.merge(df_order_products)
```

luego ya sobre este podemos simplemente agrupar por cada usuario y ver el promedio del valor reordered, de esta forma deberíamos obtener para cada cliente dicha proporción de producto ya pedido.

```
.groupby('user_id')['reordered'].mean()
```

Implementado! :) bien hecho Caro!

Escribe aquí tus conclusiones

[C5] ; Cuáles son los 20 principales artículos que las personas ponen